

To summarize, the perceptron will output -1 if the weighted sum is less than zero, and otherwise it will output 1 . Written as an equation, we have the following:

$$y = f(z), \text{ where}$$

$$z = \sum_{i=0}^n w_i x_i$$

$$f(z) = \begin{cases} -1, & z < 0 \\ 1, & z \geq 0 \end{cases}$$

$$x_0 = 1 \text{ (bias term)}$$

We note how the bias term x_0 is special in that it always is assigned the value 1. Its corresponding weight w_0 is treated just like any other weight. Code Snippet 1-1 implements this function programmatically in Python. The first element of x represents the bias term and thus must be set to 1 by the caller of the function.

Code Snippet 1-1 Python Implementation of Perceptron Function

```
# First element in vector x must be 1.  
# Length of w and x must be n+1 for neuron with n inputs.  
def compute_output(w, x):  
    z = 0.0  
    for i in range(len(w)):  
        z += x[i] * w[i] # Compute sum of weighted inputs  
    if z < 0: # Apply sign function  
        return -1  
    else:  
        return 1
```

Posterior is Useful

- Our interest in the posterior density is primarily to allow us to take all the uncertainty in $\mathbf{w}$ into account when making predictions.
- We do this by averaging over all potential values of $\mathbf{w}$ weighted by their posterior probabilities:

$$\begin{aligned} P(T_{\text{new}} = 1 | \mathbf{x}_{\text{new}}, \mathbf{X}, \mathbf{t}, \sigma^2) &= \mathbf{E}_{p(\mathbf{w} | \mathbf{X}, \mathbf{t}, \sigma^2)} \{P(T_{\text{new}} = 1 | \mathbf{x}_{\text{new}}, \mathbf{w})\} \\ &= \int P(T_{\text{new}} = 1 | \mathbf{x}_{\text{new}}, \mathbf{w}) p(\mathbf{w} | \mathbf{X}, \mathbf{t}, \sigma^2) d\mathbf{w} \end{aligned}$$

The second integral

Sampling Technique

- It enables us to cut out the approximation step and sample directly from the posterior. Recall that we are attempting to sample from $p(\mathbf{w}|\mathbf{t}, \mathbf{X}, \sigma^2)$ so that we can approximate

$$\begin{aligned} P(T_{\text{new}} = 1 | \mathbf{x}_{\text{new}}, \mathbf{X}, \mathbf{t}, \sigma^2) &= \mathbf{E}_{p(\mathbf{w}|\mathbf{X}, \mathbf{t}, \sigma^2)} \{P(T_{\text{new}} = 1 | \mathbf{x}_{\text{new}}, \mathbf{w})\} \\ &= \int P(T_{\text{new}} = 1 | \mathbf{x}_{\text{new}}, \mathbf{w}) p(\mathbf{w} | \mathbf{X}, \mathbf{t}, \sigma^2) d\mathbf{w} \end{aligned}$$

with:

$$P(T = 1 | \mathbf{x}_{\text{new}}, \mathbf{X}, \mathbf{t}, \sigma^2) \simeq \frac{1}{N_s} \sum_{s=1}^{N_s} P(T_{\text{new}} = 1 | \mathbf{x}_{\text{new}}, \mathbf{w}_s).$$

Metropolis-Hastings Algorithm

- Metropolis-Hastings generates a sequence of samples in two steps until we have collected enough.
 1. We need to *propose* a new sample a candidate for $\mathbf{w}_s$, which is performed by proposing a movement from the previous sample $\mathbf{w}_{s-1}$.
 2. The proposed sample is tested to see whether or not it should be accepted. If accepted, it becomes our new sample $\mathbf{w}_s$. If not accepted, our new sample is equal to the previous one, $\mathbf{w}_s = \mathbf{w}_{s-1}$.

A Word of Caution

- The initial point $\mathbf{w}_1$ does not matter, and the sampler is guaranteed to converge in theory.
- In practice, it is important to use one (or ideally more) of the methods available to test convergence before we start harvesting samples.
- Checks for unrepresentativeness usually look for lingering influence of the initial value and for orphaned chains that have somehow settled into unusual regions of parameter space. (Puppy §7.5.1)

Proposing a New Sample

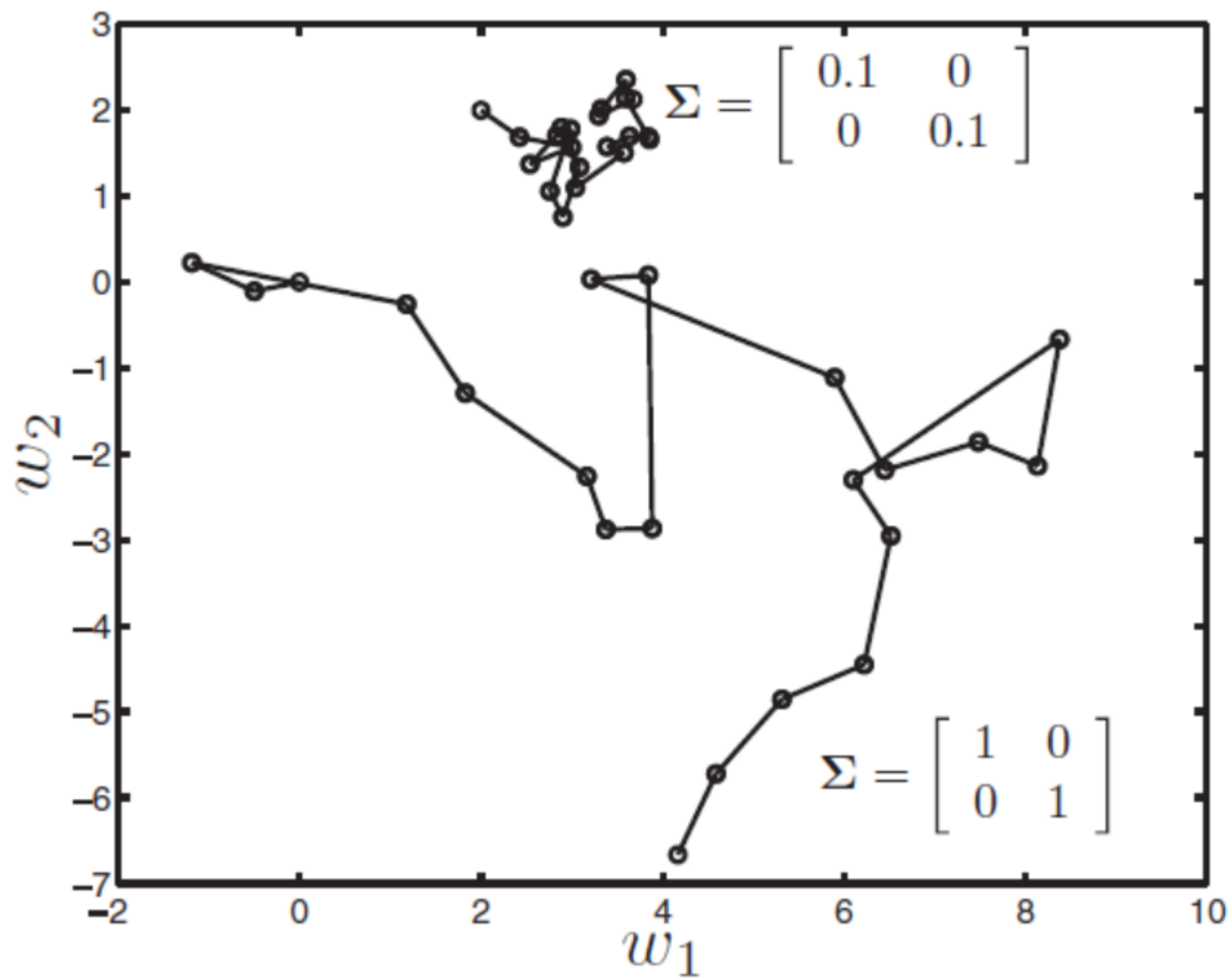
- Assume that we have already sampled $s - 1$ values using the MH scheme. Based on a movement from $\mathbf{w}_{s-1}$, we will draw a proposed sample $\widetilde{\mathbf{w}}_s$ from

$$p(\widetilde{\mathbf{w}}_s | \mathbf{w}_{s-1})$$

- The proposal density does not have to have any connection with the posterior $p(\mathbf{w} | \mathbf{t}, \mathbf{X}, \sigma^2)$. In practice, the proposal density will have an impact on how long it will take the MH sampler to converge.
- A common choice is $p(\widetilde{\mathbf{w}}_s | \mathbf{w}_{s-1}, \boldsymbol{\Sigma}) = N(\mathbf{w}_{s-1}, \boldsymbol{\Sigma})$.

Random Walk

- Sampling a sequence of values like this creates what is known as a random walk.
- Two such walks are shown in the following figure. One starts with $\mathbf{w}_1 = [0, 0]^T$ and has covariance $\Sigma = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ whilst the other starts from $\mathbf{w}_1 = [2, 2]^T$ and has covariance $\Sigma = \begin{bmatrix} 0.1 & 0 \\ 0 & 0.1 \end{bmatrix}$.
- The proposal is 1. easy to sample from; 2. symmetric $p(\widetilde{\mathbf{w}}_s | \mathbf{w}_{s-1}, \Sigma) = p(\mathbf{w}_{s-1} | \widetilde{\mathbf{w}}_s, \Sigma)$.  It is Metropolis in fact.

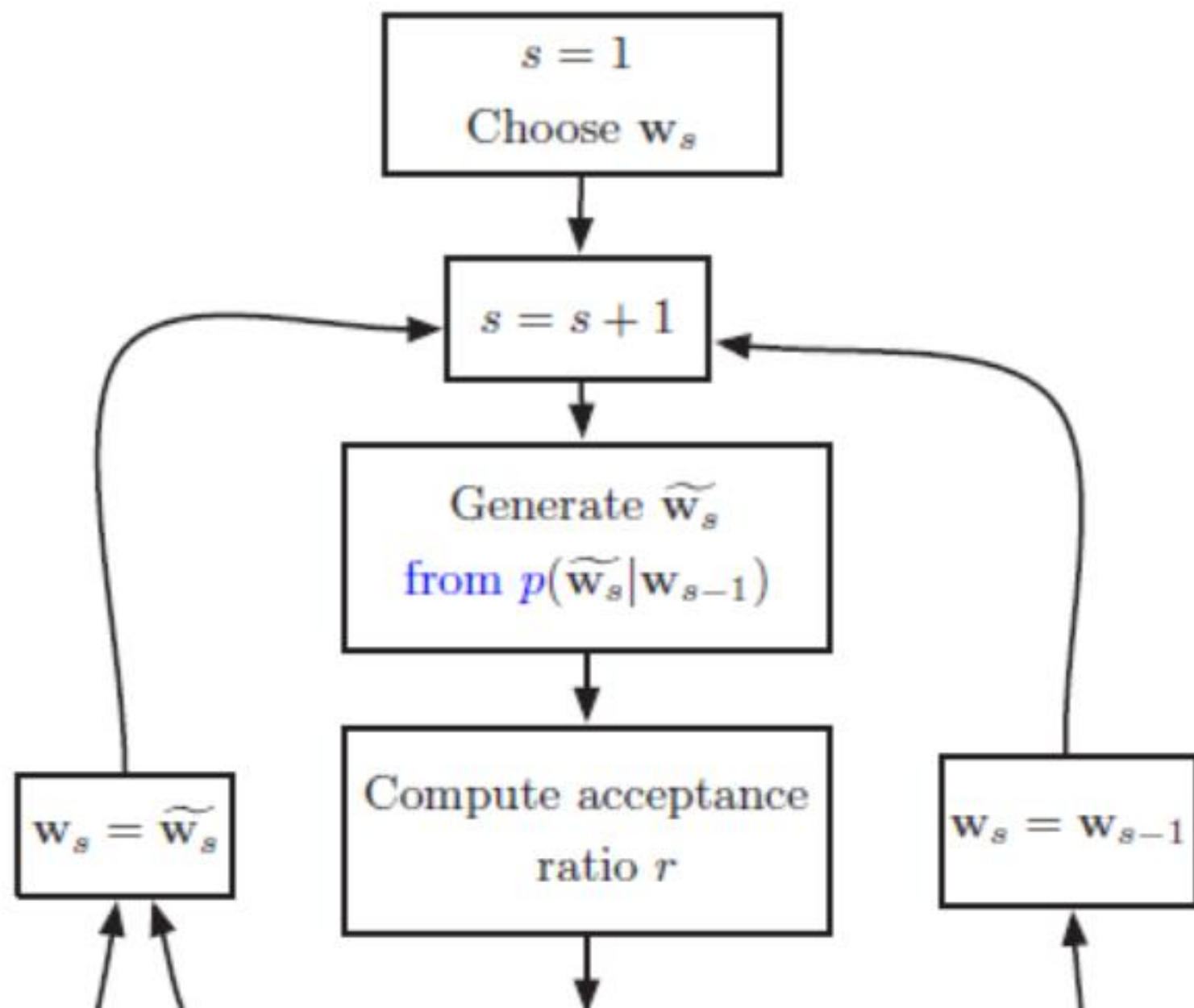


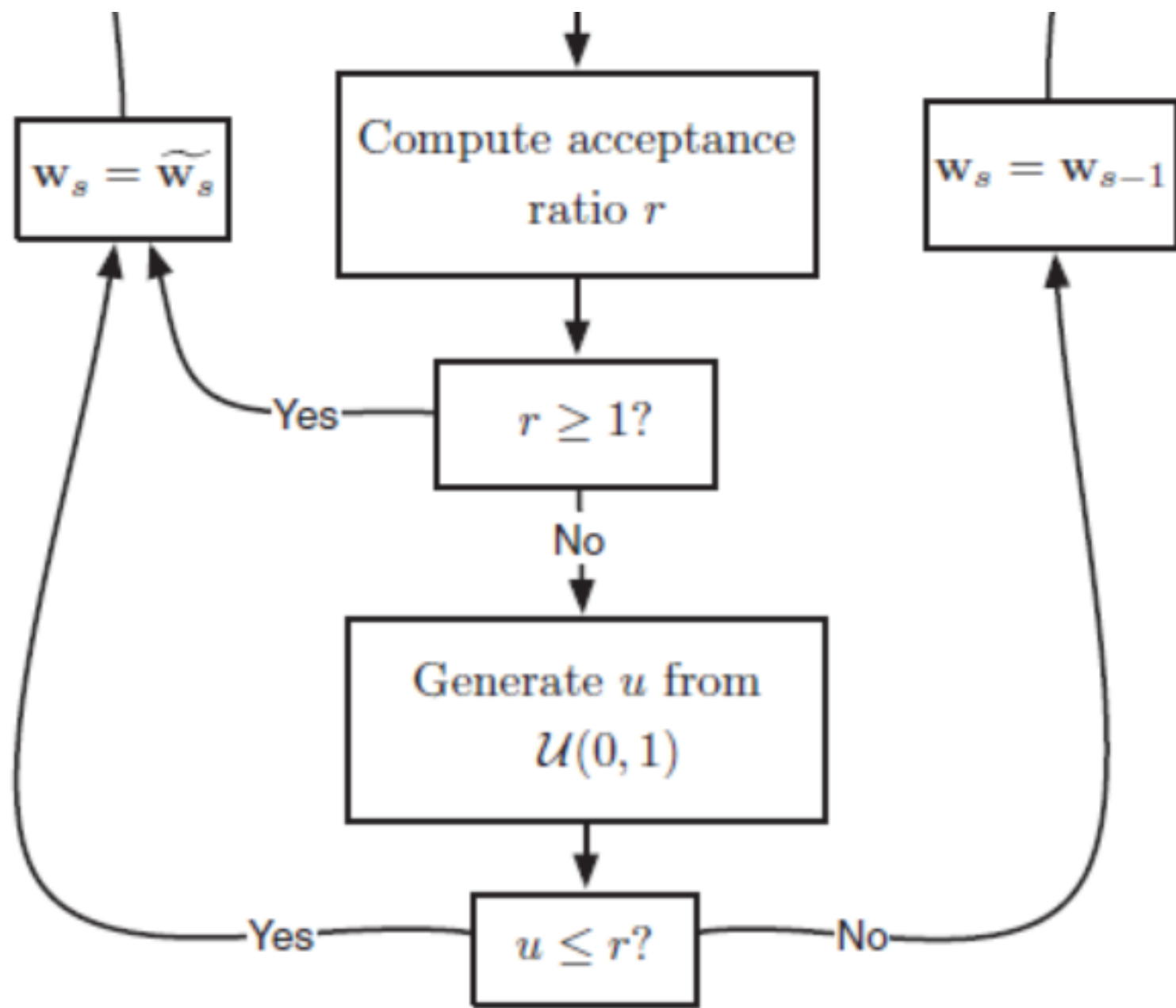
Accept/Reject Step

- We now have proposed $\widetilde{\mathbf{w}}_s$, we must decide whether we should accept it or reject it based on

$$r = \frac{p(\widetilde{\mathbf{w}}_s | \mathbf{X}, \mathbf{t}, \sigma^2) * p(\mathbf{w}_{s-1} | \widetilde{\mathbf{w}}_s, \Sigma)}{p(\mathbf{w}_{s-1} | \mathbf{X}, \mathbf{t}, \sigma^2) * p(\widetilde{\mathbf{w}}_s | \mathbf{w}_{s-1}, \Sigma)}$$

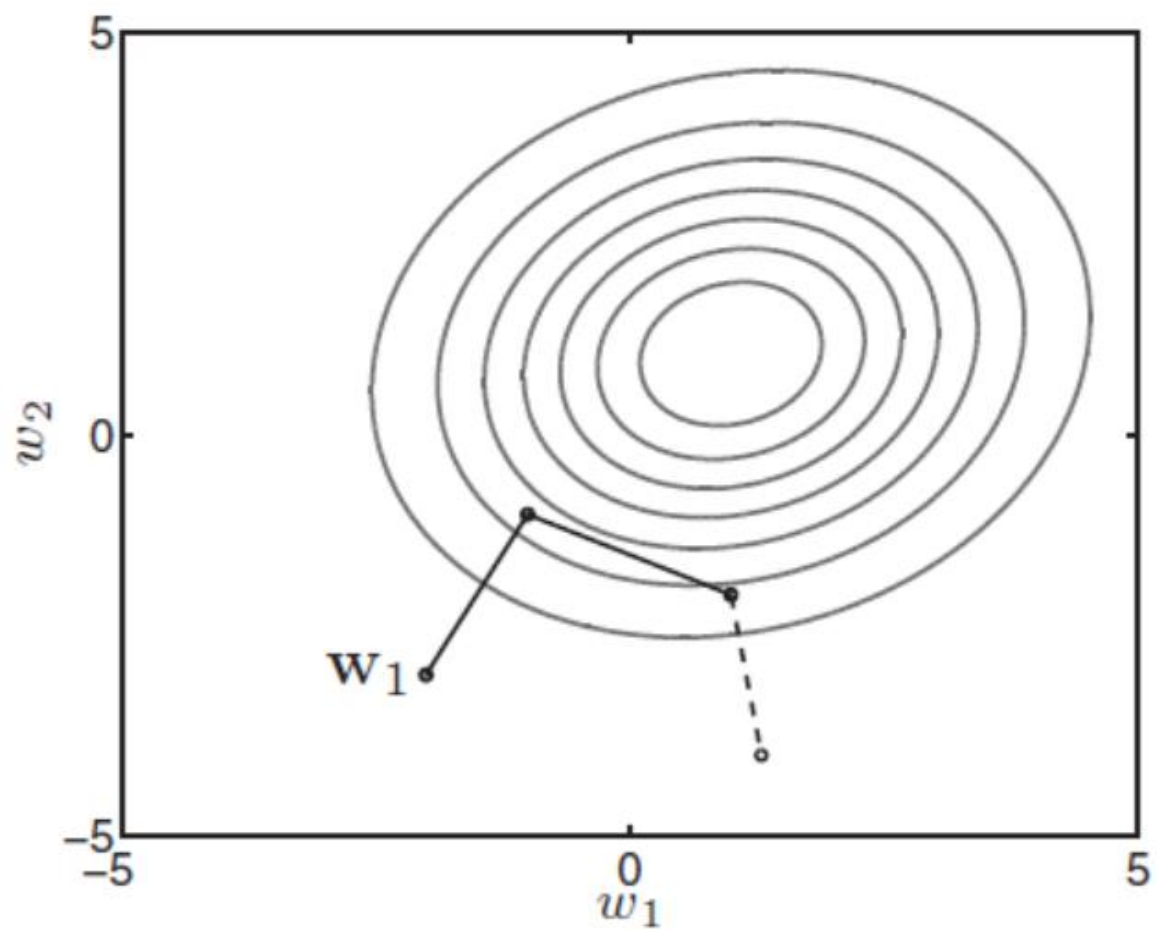
- This is the ratio of the posterior density at the proposed sample to that at the old sample multiplied by the ratio of the proposal densities.
- The symmetry of the Gaussian proposal distribution discussed above allows us to ignore this last term.



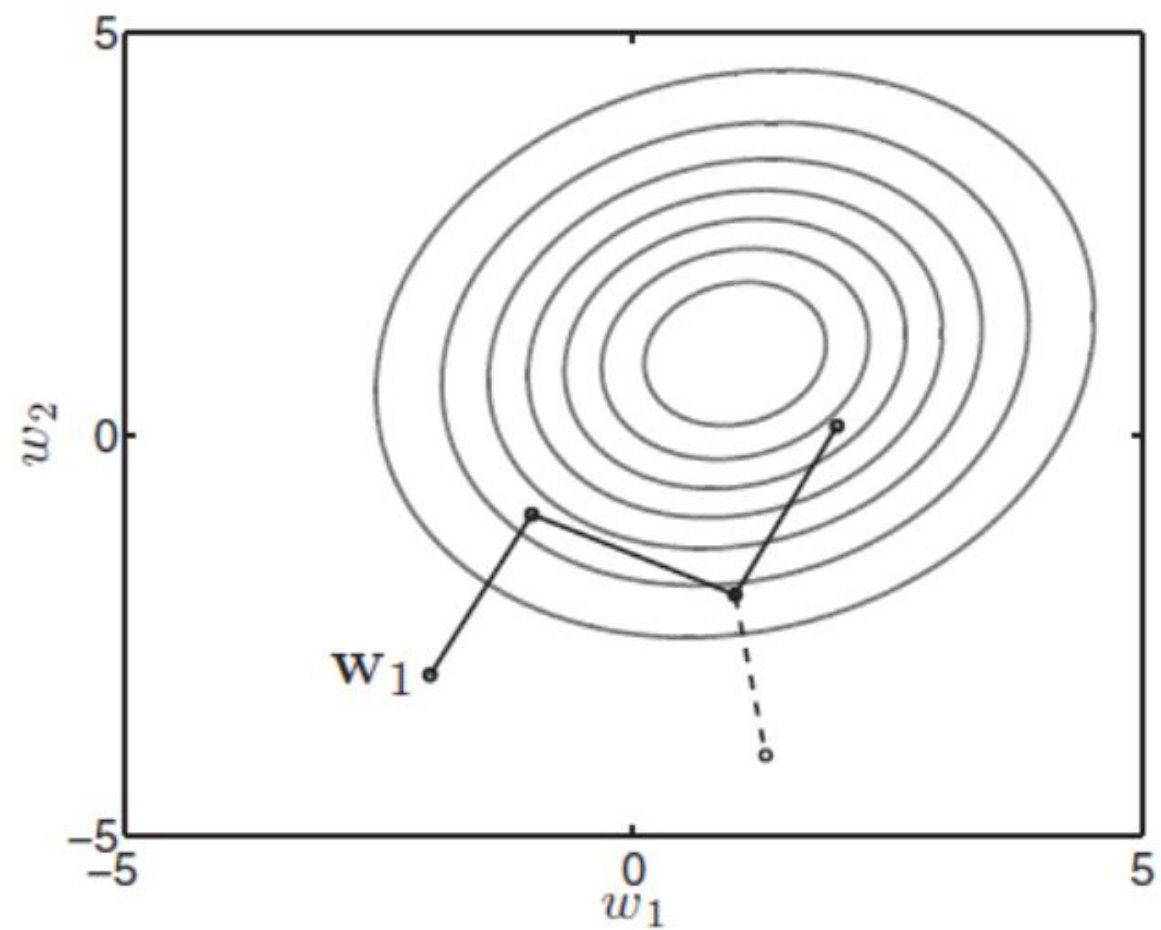


Accept/Reject Step

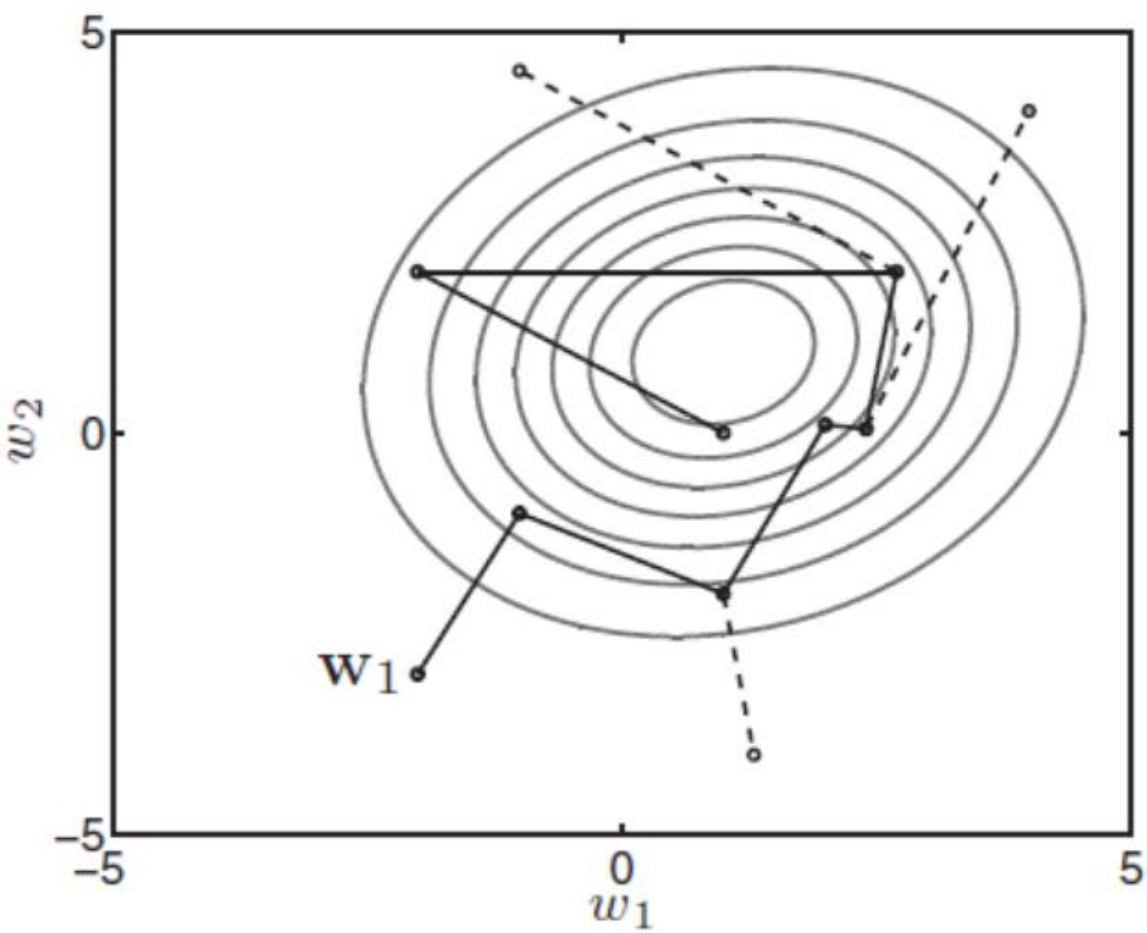
- If r is one or greater, we accept the sample ($\mathbf{w}_s = \widetilde{\mathbf{w}}_s$). If $r < 1$, we accept the sample with probability equal to r .
- Hence, we accept the proposal if a value drawn from a uniform distribution $u \leq r$ and reject otherwise.
- Our proposal density is Gaussian with $\Sigma = \mathbf{I}$. (How about the mean of proposal Gaussian?)
- How about the first proposal $\mathbf{w}_1$?



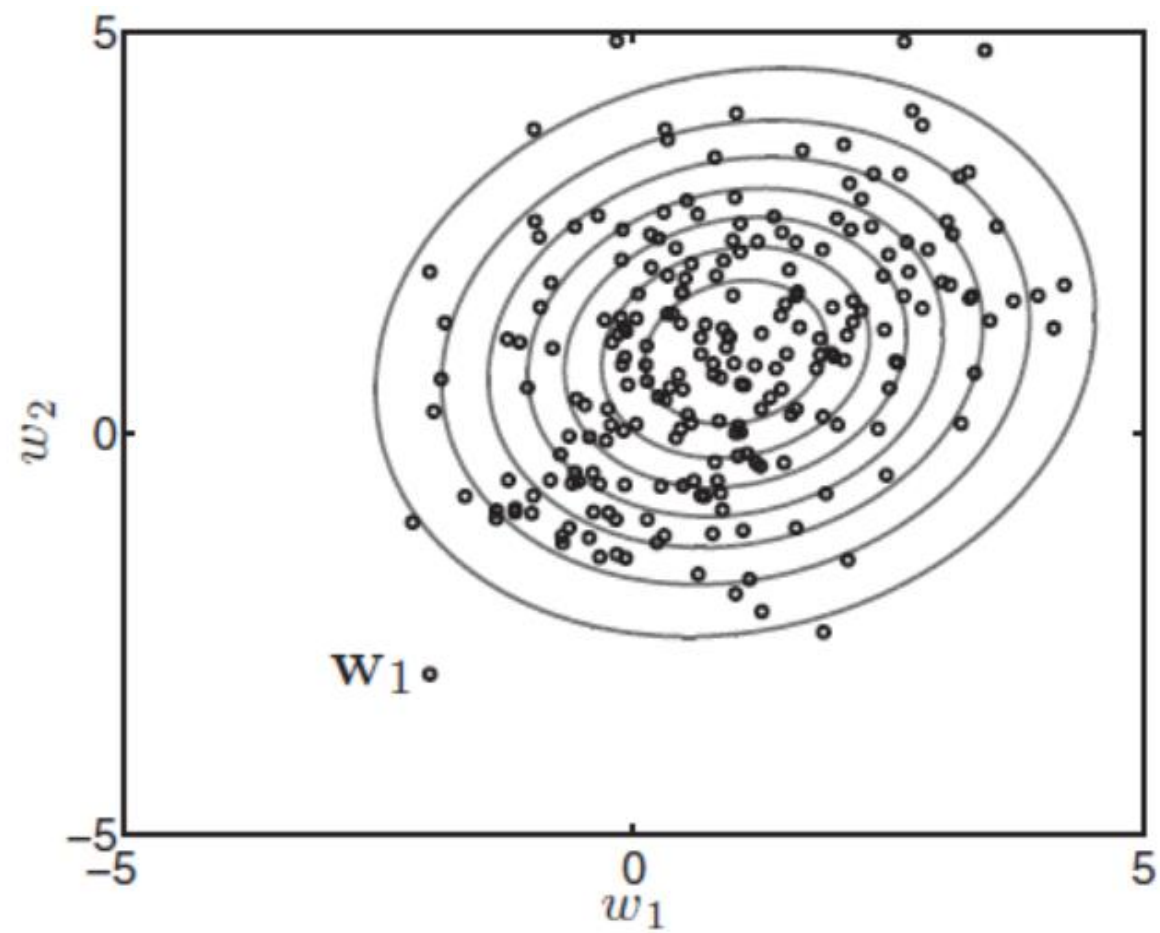
(c) After three samples. $\tilde{w}_3$ was accepted, $\tilde{w}_4$ rejected (dashed line).



(d) After four samples.



(e) After ten samples.



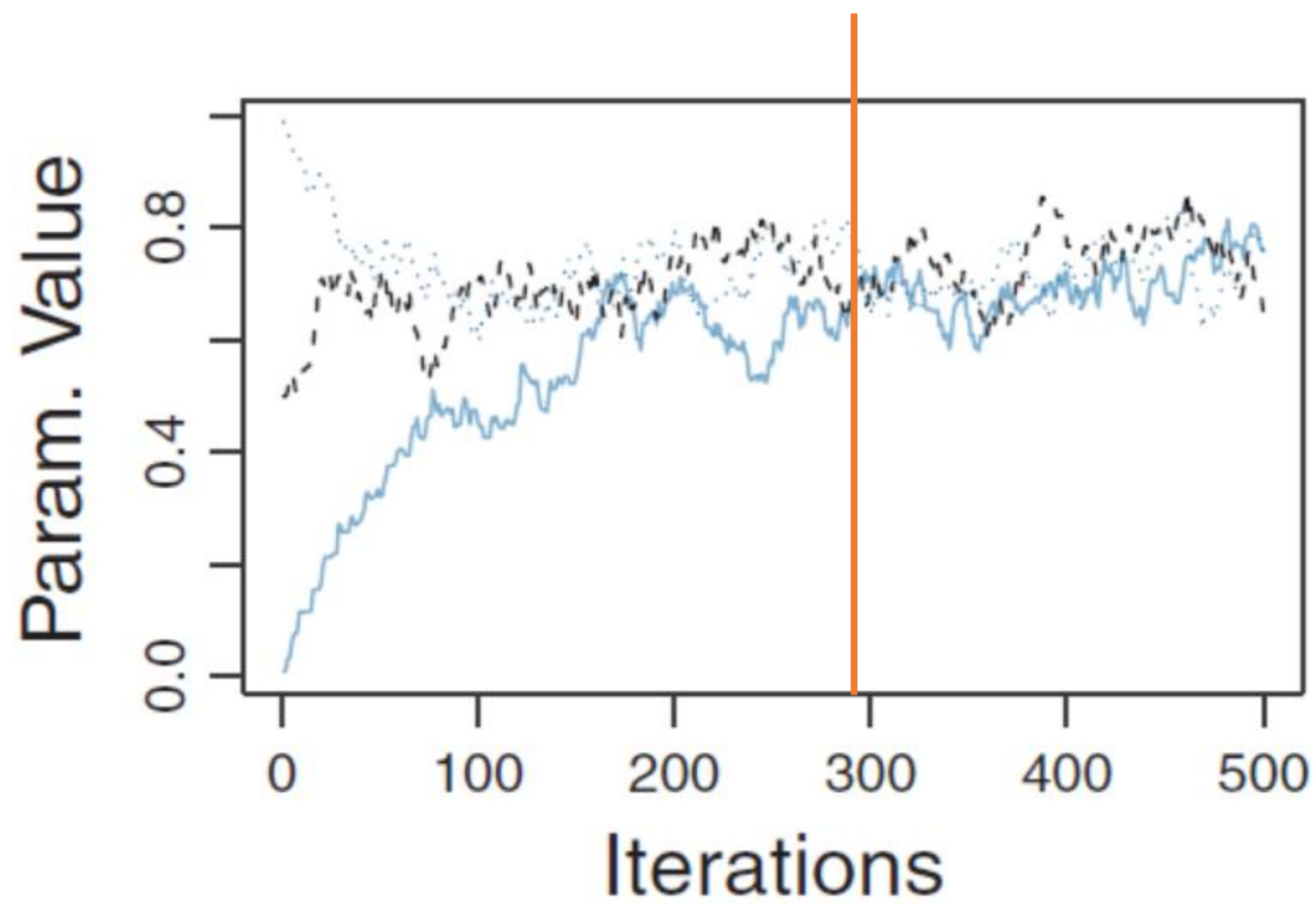
(f) The first 300 samples.

Burn-In

- We don't necessarily know if we are starting the sampler in an area of very low posterior density. Therefore, the first few samples may not be representative and should be discarded.
- The period between the starting point and convergence of the sampler is known as the *burn-in period*.
- Sadly, it is not possible to conclusively determine how long this period should be.

Convergence

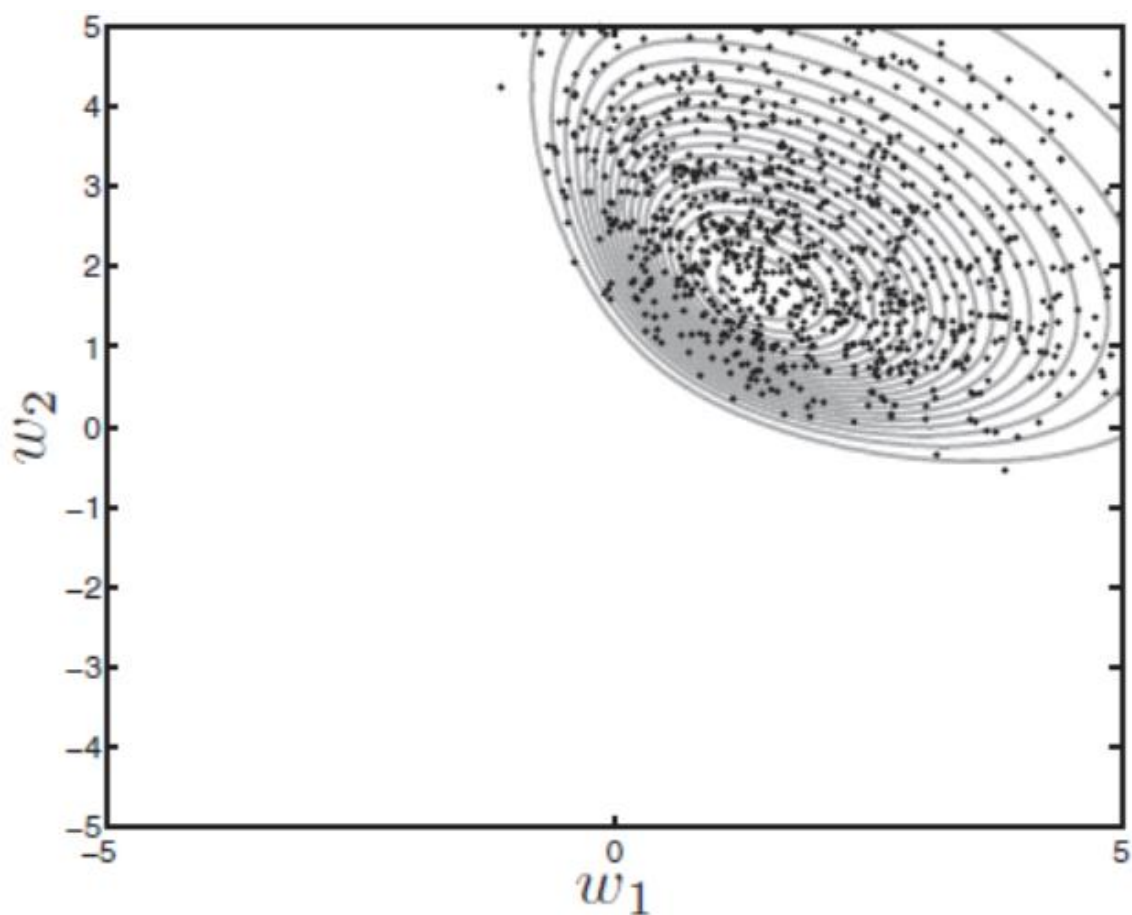
- To overcome this problem we need a method for determining convergence. This is not convergence to a particular value, but convergence *to a particular distribution*.
- A popular method is to start several samplers simultaneously from different starting points.
- When all of the samplers are generating samples with similar characteristics (mean, variance, etc.), they have all converged to the same distribution.



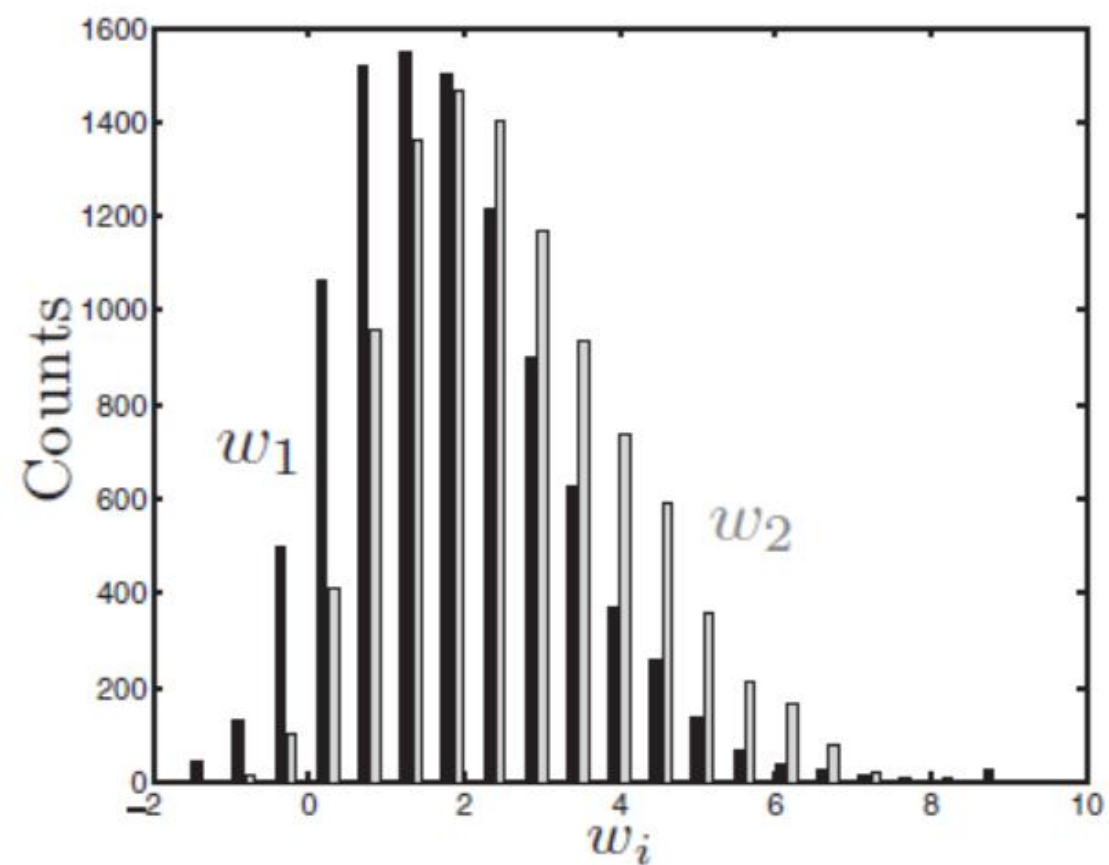
Marginal Distribution

- Given that $\Sigma = 0.5^2 * \mathbf{I}$, we show every tenth sample along with the posterior contours.
- To marginalize w_2 from the posterior we would need to integrate (or sum, if the random variable is discrete) over all values w_2 could take:

$$\begin{aligned} p(w_1 | \mathbf{X}, \mathbf{t}, \sigma^2) &= \int p(\mathbf{w} | \mathbf{X}, \mathbf{t}, \sigma^2) dw_2 \\ &= \int p(w_1, w_2 | \mathbf{X}, \mathbf{t}, \sigma^2) dw_2 \end{aligned}$$



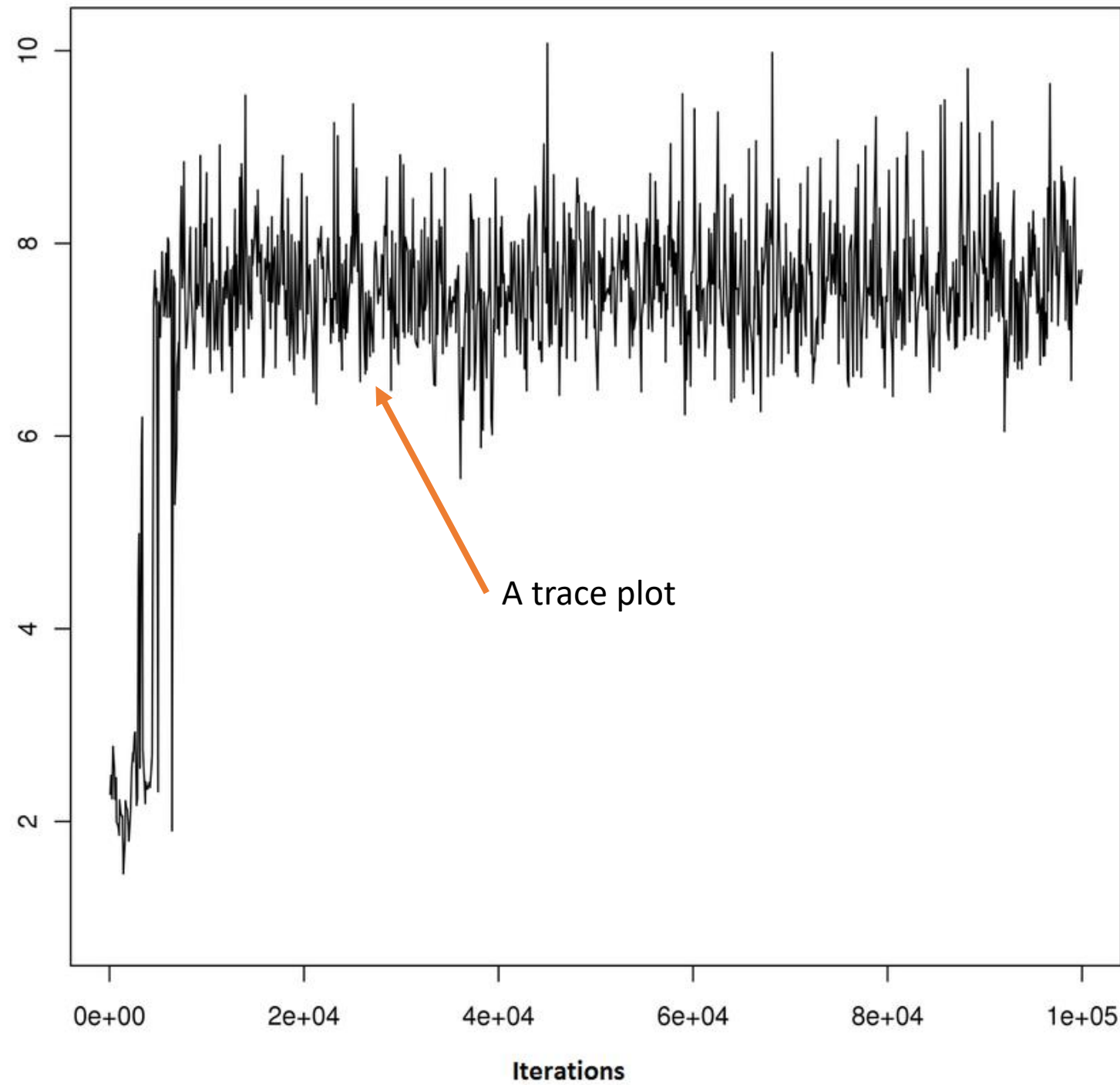
(a) One thousand of the MH samples along with the posterior contours.

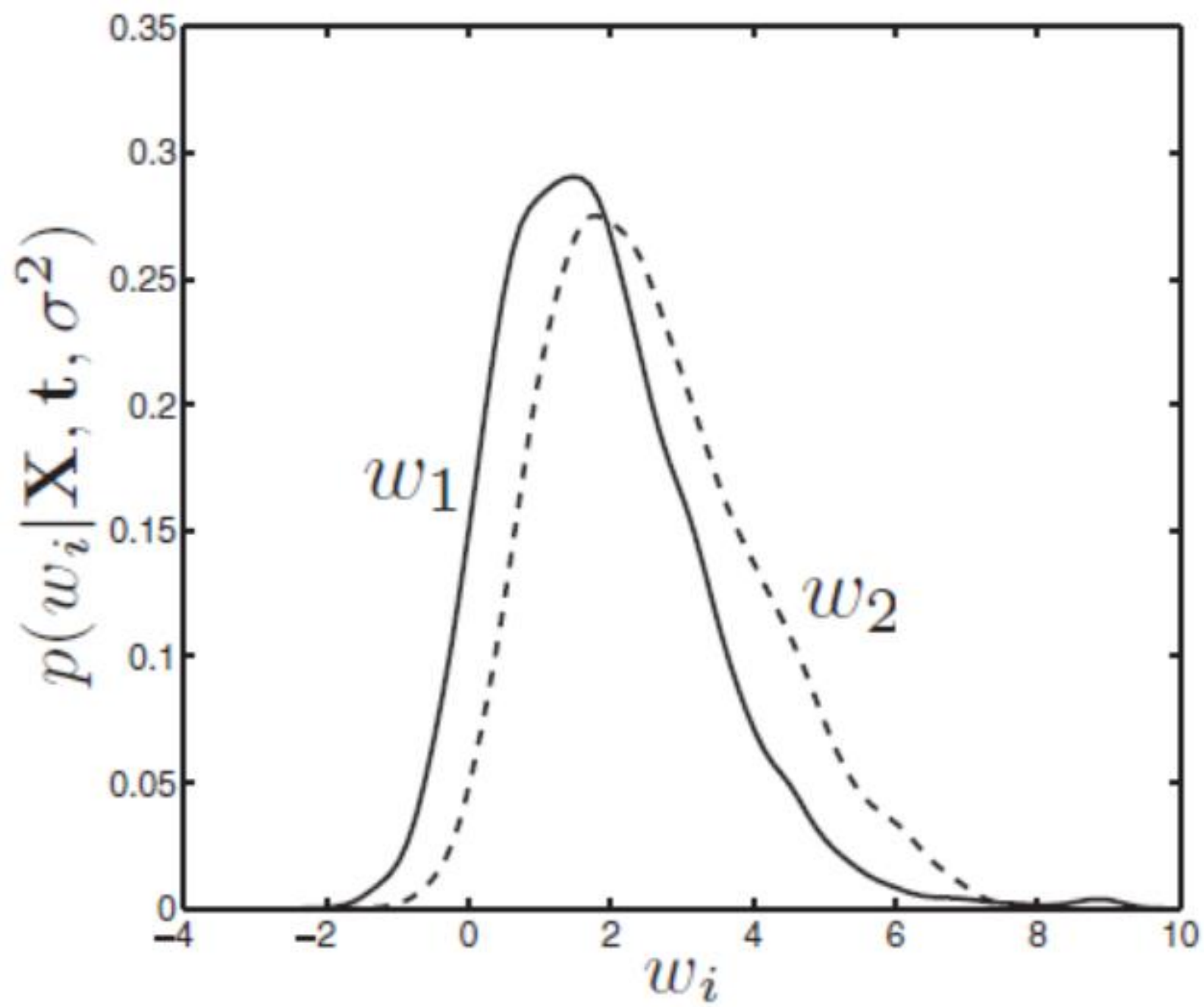


(b) Histograms of the samples for both w_1 (black) and w_2 (grey).

Visualize Marginal Samples

- if we throw away the value of w_2 from each sample, we are left with a set of samples from $p(w_1 | \mathbf{X}, \mathbf{t}, \sigma^2)$.
- 1. Relative frequency histograms could be thought of as the posterior probabilities that the w_1 (or w_2) falls into each of these bins.
- 2. The *trace plot* for w_i gives us confidence that the sampler has converged very quickly. If it hadn't, we might see an overall increasing or decreasing trend.





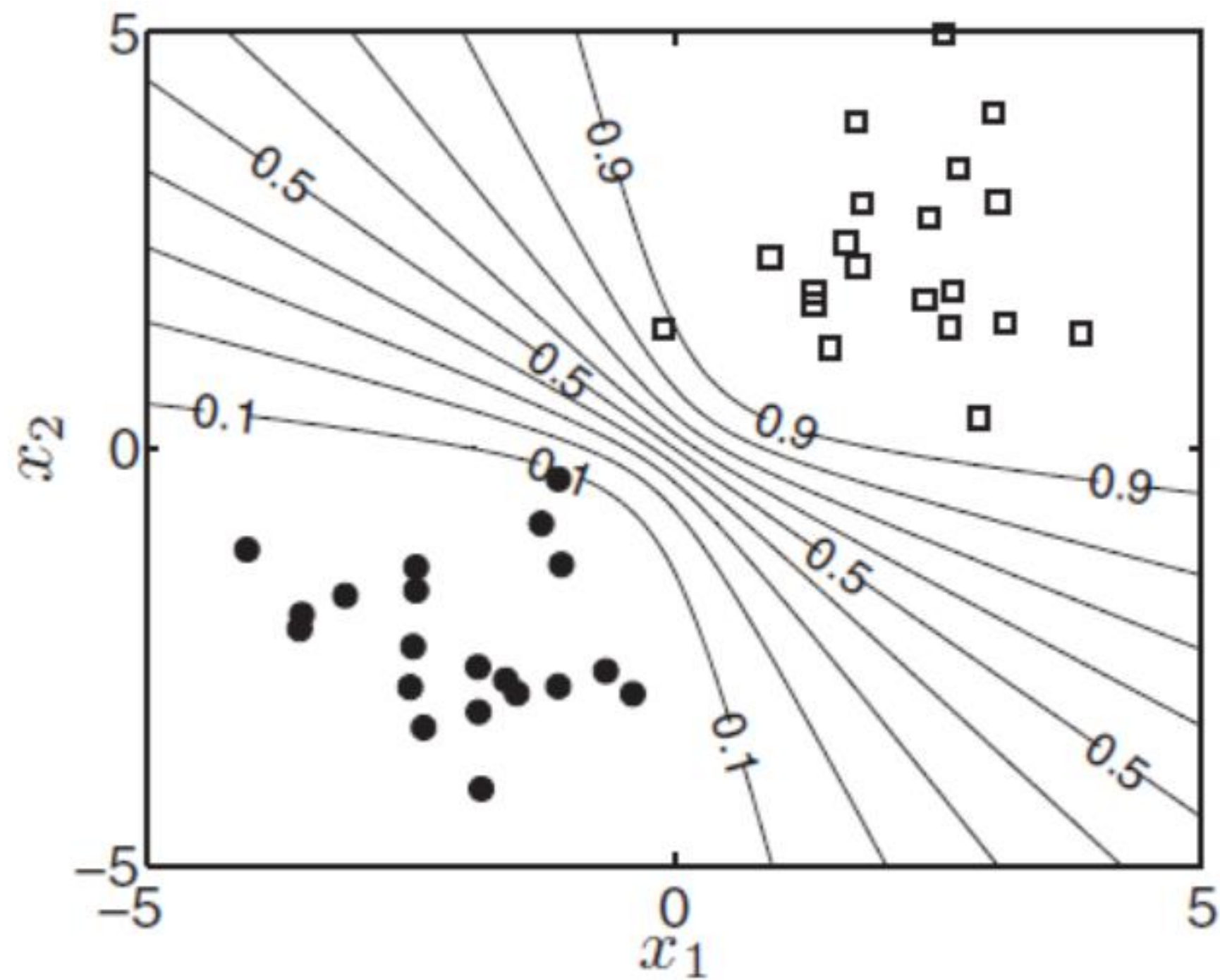
Predictive Probability

- We now have a set of samples from the true $p(\mathbf{w}|\mathbf{X}, \mathbf{t}, \sigma^2)$, and we can substitute them into

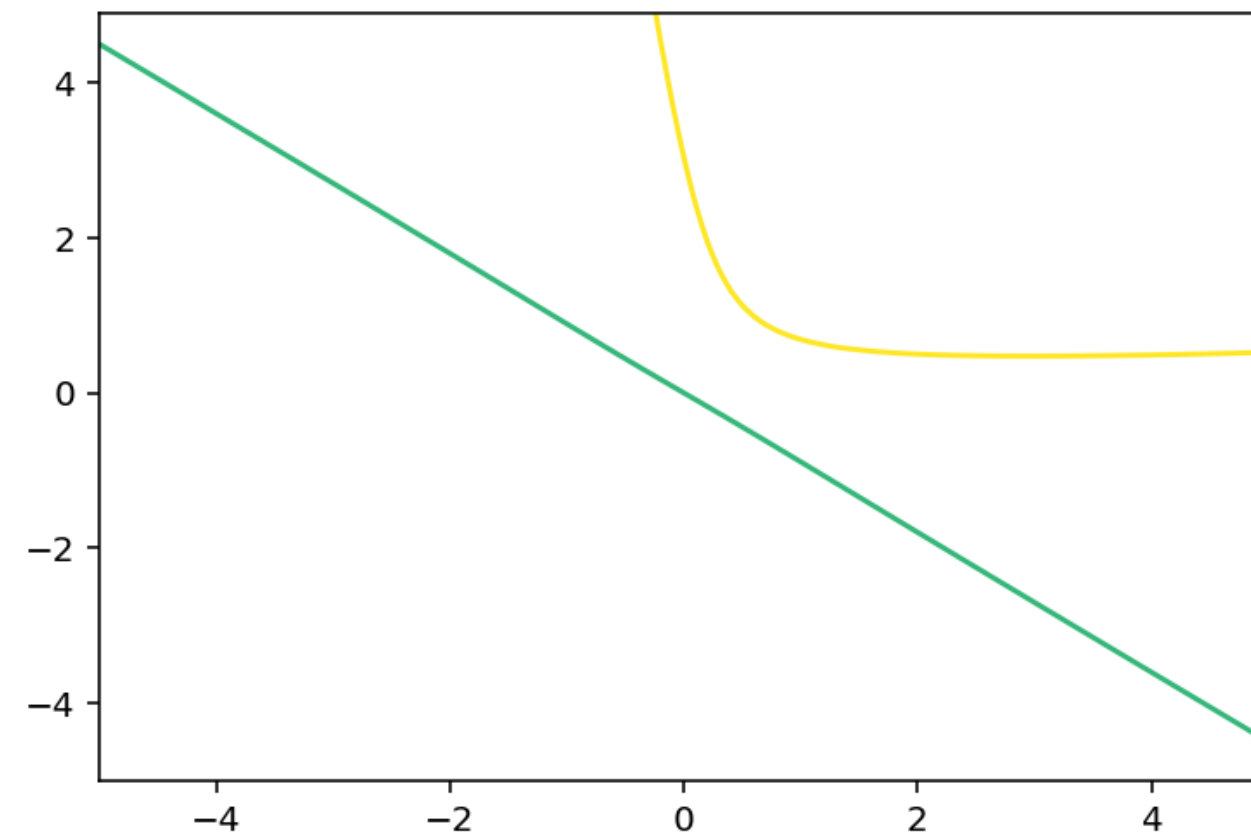
$$P(T_{\text{new}} = 1 | \mathbf{x}_{\text{new}}, \mathbf{X}, \mathbf{t}, \sigma^2) = \frac{1}{N_s} \sum_{s=1}^{N_s} \frac{1}{1 + \exp(-\mathbf{w}_s^T \mathbf{x}_{\text{new}})}$$

and compute the predictive probability contours.

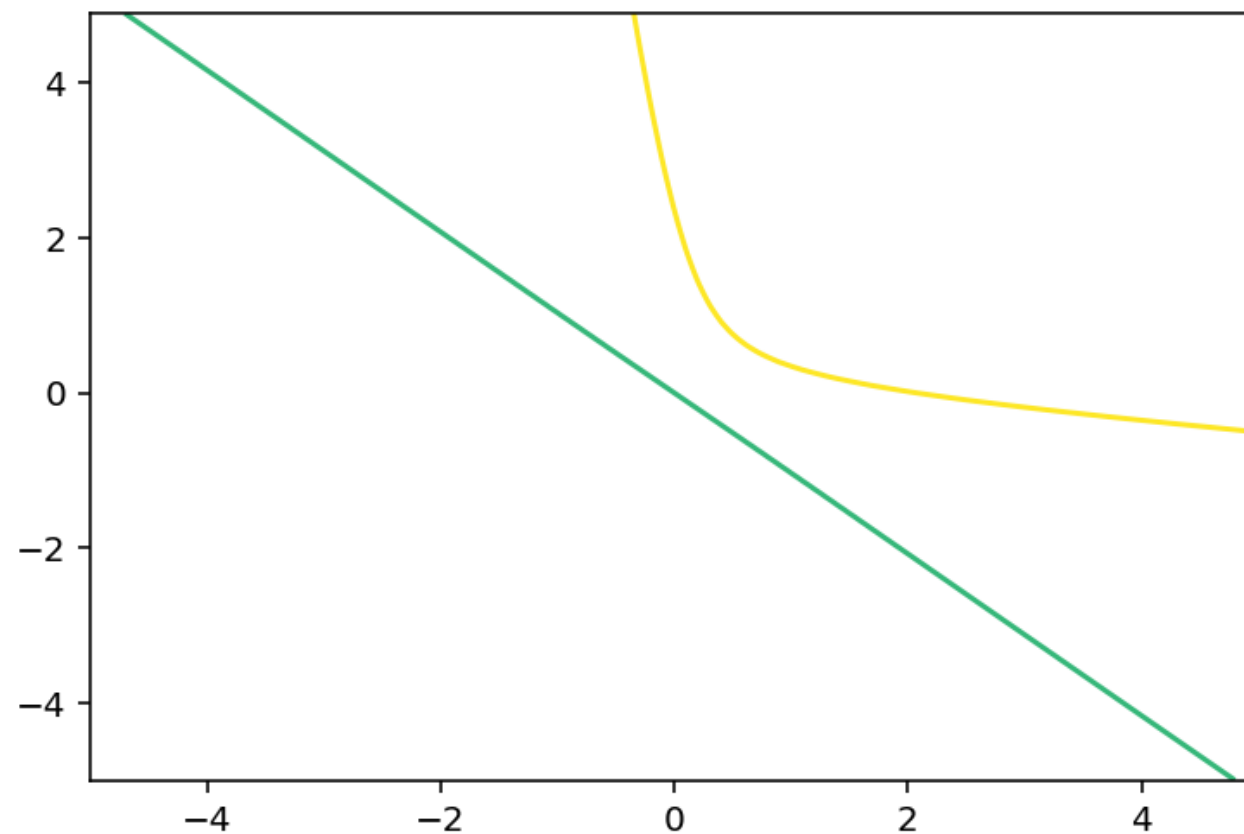
- One noticeable difference is that the contours are slightly *less tightly curved* around the areas in which the data lie.



Contours by Metropolis sampling



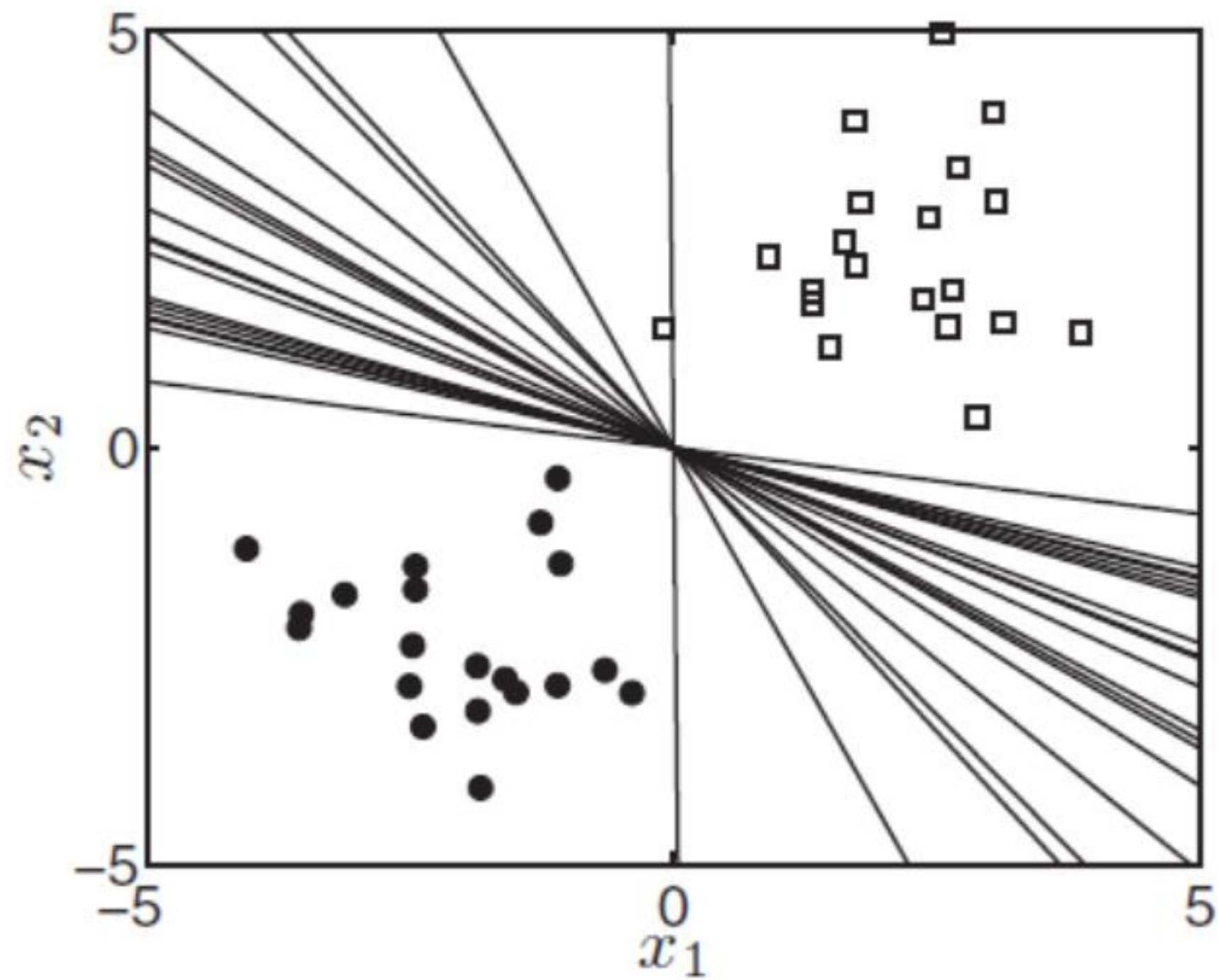
Sampling directly from the posterior



Interpretation

- This suggests that the probability reduces rather more slowly as we move away from the squares.
- The MH sampler is sampling from the true posterior and so the contours in Figure 4.12(e) *should be considered closer to the truth* than those for the Laplace approximation in Figure 4.6(a).

For approximation rather than for inference.
- The decision boundaries corresponding to 20 of the MH samples picked at random are presented in the following figure.



Caveat 1

- The Metropolis-Hastings algorithm will not always work well for any model. The difficulty lies in the (often unknown) shape of the density from which we are attempting to sample.
- For example, when density has two modes. In theory, we will move from the top of one mode to the top of the other at some point *eventually*.
- In practice, we might get very old while we wait.